Deletion in Circular Doubly Linked List (CDLL)

Types of Deletion

1. **Deletion at Beginning**
2. **Deletion at End**
3. **Deletion at Specific Position**

Program in C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};

/* Delete at Beginning */
struct Node* deleteBegin(struct Node *head)
{
    struct Node *temp, *last;

    if(head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    // Only one node
    if(head->next == head)
    {
        free(head);
        return NULL;
    }

    last = head->prev;
    temp = head;

    head = head->next;

    last->next = head;
    head->prev = last;
```

```
    free(temp);
    return head;
}

/* Delete at End */
struct Node* deleteEnd(struct Node *head)
{
    struct Node *temp, *last;

    if(head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    // Only one node
    if(head->next == head)
    {
        free(head);
        return NULL;
    }

    last = head->prev;
    temp = last;

    last->prev->next = head;
    head->prev = last->prev;

    free(temp);
    return head;
}

/* Delete at Specific Position */
struct Node* deletePosition(struct Node *head)
{
    struct Node *temp;
    int pos, i;

    if(head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    printf("Enter position: ");
    scanf("%d", &pos);

    // Delete first node
    if(pos == 1)
```

```
    {
        return deleteBegin(head);
    }

    temp = head;

    for(i = 1; i < pos; i++)
    {
        temp = temp->next;

        if(temp == head)
        {
            printf("Invalid Position\n");
            return head;
        }
    }

    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;

    free(temp);
    return head;
}

/* Display CDLL */
void display(struct Node *head)
{
    struct Node *temp;

    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

    printf("CDLL: ");
    do
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    while(temp != head);

    printf("(back to head)\n");
}
```

```
/* Insert at End (for testing) */
struct Node* insertEnd(struct Node *head, int value)
{
    struct Node *newnode, *last;

    newnode = (struct Node*)malloc(sizeof(struct Node));
    newnode->data = value;

    if(head == NULL)
    {
        newnode->next = newnode;
        newnode->prev = newnode;
        return newnode;
    }

    last = head->prev;

    newnode->next = head;
    newnode->prev = last;

    last->next = newnode;
    head->prev = newnode;

    return head;
}

/* Main Function */
int main()
{
    struct Node *head = NULL;
    int choice;

    // Initial creation
    head = insertEnd(head, 10);
    head = insertEnd(head, 20);
    head = insertEnd(head, 30);

    while(1)
    {
        printf("\n----- MENU -----\n");
        printf("1. Delete at Beginning\n");
        printf("2. Delete at End\n");
        printf("3. Delete at Position\n");
        printf("4. Display\n");
        printf("5. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);
```

```
        switch(choice)
        {
            case 1:
                head = deleteBegin(head);
                break;

            case 2:
                head = deleteEnd(head);
                break;

            case 3:
                head = deletePosition(head);
                break;

            case 4:
                display(head);
                break;

            case 5:
                exit(0);

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Explanation (Short)

Delete at Beginning

- Move head to next node
- Update last node (`head->prev`)
- Maintain circular links

Delete at End

- Use `head->prev` to find last node
- Update second last node
- Maintain circular links

Delete at Position

- Traverse to required node
- Adjust both `prev` and `next` pointers

- Handle invalid positions

Sample Output

```
CDLL: 10 <-> 20 <-> 30 <-> (back to head)

Delete at Beginning
CDLL: 20 <-> 30 <-> (back to head)

Delete at End
CDLL: 20 <-> (back to head)
```